

# MagicCode Super-Resolution Software Specification

Document Version: v2.0.0. Release Date: September 2, 2026.

## 1. Document Overview

This specification defines capabilities, platforms, and functional boundaries of MagicCode Super-Resolution Software v2.0.0. It covers spatial and temporal GPU paths and spatial CPU paths.

### 1.4 Document Status

| Version | Date              | Status  | Change                                                 |
|---------|-------------------|---------|--------------------------------------------------------|
| v2.0.0  | September 2, 2026 | Updated | Temporal modes, session-API contract, platform matrix. |

## 2. System Requirements

| Platform             | GPU backends                    | Temporal                          |
|----------------------|---------------------------------|-----------------------------------|
| Android              | Vulkan, OpenGL ES               | Temporal GPU yes; CPU temporal no |
| iOS / iPadOS / macOS | Metal                           | Temporal GPU yes; CPU temporal no |
| Windows              | Vulkan, Direct3D 11, OpenGL 4.3 | Temporal GPU yes; CPU temporal no |

- Input size [64, 4032]. Spatial scale [1, 8]. Temporal scale (1, 8].
- CPU spatial: x86\_64 AVX2 and ARM64 NEON buffer paths. CPU temporal is not available.

## 3. Functional Capabilities

| Value | Enumerator             | Role                                |
|-------|------------------------|-------------------------------------|
| 0     | SPATIAL_SPEED_MODE     | Spatial, throughput first           |
| 1     | SPATIAL_BALANCED_MODE  | Spatial, quality/cost trade-off     |
| 2     | TEMPORAL_SPEED_MODE    | Temporal, throughput first          |
| 3     | TEMPORAL_BALANCED_MODE | Temporal, canonical FSR-family path |

Spatial modes upscale a single frame. Temporal modes accumulate history and require depth, motion, jitter, and camera parameters through magic\_frame\_t / temporal\_frame\_t.

## Temporal Super-Resolution

v2.0.0 adds temporal super-resolution through the session API: libmagic\_sr.a (Windows: libmagic\_sr.lib) + mc\_interface.h + MC\_Init / MC\_Process / MC\_Uninit. MC\_Enable remains a single-texture spatial helper. It may accept a temporal alg\_mode\_e value by range check, but it cannot supply per-frame depth, motion, or jitter and must not be documented as a complete temporal entry.

### Modes

| Value | Enumerator             | Role                                |
|-------|------------------------|-------------------------------------|
| 0     | SPATIAL_SPEED_MODE     | Spatial, throughput first           |
| 1     | SPATIAL_BALANCED_MODE  | Spatial, quality/cost trade-off     |
| 2     | TEMPORAL_SPEED_MODE    | Temporal, throughput first          |
| 3     | TEMPORAL_BALANCED_MODE | Temporal, canonical FSR-family path |

- Temporal scaler\_factor is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- TEMPORAL\_SPEED\_MODE on Metal, Vulkan, and OpenGLS currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- TEMPORAL\_BALANCED\_MODE uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGLS do not run edge-resolve. If enable\_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

### Minimum per-frame inputs

- magic\_frame\_t.image\_in / image\_out: caller-owned color and output GPU resources.
- magic\_frame\_t.frame: non-NULL temporal\_frame\_t\* with struct\_size = sizeof(temporal\_frame\_t).
- temporal\_frame\_t.depth and motion at input resolution.
- jitter\_offset\_x/y in input pixels, top-left, +X right, +Y down.
- frame\_index / reset\_history; camera\_near / camera\_far / camera\_fov\_y (radians).
- gpu\_context at MC\_Init: Vulkan requires physical\_device + device; D3D11 requires device; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan temporal requires magic\_frame\_t.command\_buffer to be a recording VkCommandBuffer. Metal command\_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth\_reversed / depth\_infinite / hdr\_color on input\_param\_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

### Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.

- `motion_vector_scale_x/y` is the only MV numeric conversion. (0,0) defaults to (input\_width, input\_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: `MAGICSR_TAAU_AUTO_MASK=0` and `MAGICSR_TAAU_HIST_REJECT=0` can turn those defaults off. They are not a customer configuration path.

## 4. Resource and synchronization contract

- Caller owns application GPU objects. The library owns internal history, scratch, and pipelines only.
- Vulkan: library does not begin/end/submit/wait. Restore of input layouts after compute; output remains GENERAL.
- Complete prior GPU work before `MC_Uninit` or a size-changing `SET_PARAM`.

## 5. Engine plugins

Unity and Unreal plugins expose Enable/Disable for spatial GPU textures. Full temporal integration is the Native session API described above. Plugin Enable is not a complete temporal entry.

## 6. Conformance notes

API support is not the same as device-run verification. Exact 2× has optimized temporal kernels; other rates in (1, 8] are supported by the API. Do not claim every scale or GPU vendor is verified.